SentientGate

AI-Powered Runtime Security Fabric for Cloud-Native Microservices

A resilient, distributed security mesh engineered to sit at the microservice edge, providing sub-millisecond request enforcement, out-of-band Kafka telemetry streaming, binary gRPC log analysis, heuristic strategy engines, and on-premise LLM anomaly detection.

Java 21 | Spring WebFlux | Apache Kafka | Redis 7 | Ollama LLM | gRPC | PostgreSQL 16 | React 18 | Kubernetes / Helm | Terraform EKS

System Architecture & Technical Scenarios Report

Edge Enforcement • Concurrency Isolation • Threat Strategies • Local AI • Cloud Infrastructure

Author: **Shrihari R. Kulkarni**

Version: 2.0 (Production Release)

License: Apache 2.0

Repository: SentientGate-AI-Powered-Runtime-Security

Contents

1 Executive Summary & System Scenarios **2**
1.1 The Architectural Vision . . . 2
1.2 Production Service Inventory . . . 3

2 End-to-End Architectural Lifecycle **4**
2.1 Request & Threat Detection Lifecycle . . . 4
2.2 Concurrency & Thread Isolation Model . . . 5

3 Microservice Component Specifications **6**
3.1 ApiGateway: Non-Blocking Edge Mesh . . . 6
3.1.1 Filter Chain Execution Order . . . 6
3.1.2 Management & Metrics Controllers . . . 6
3.2 MCPService: Malicious Client Protection Engine . . . 7
3.2.1 Layered Heuristic Strategies . . . 7
3.2.2 Resilience & Fault Tolerance . . . 7
3.3 LoggingService: Persistence & gRPC History Provider . . . 7
3.3.1 Storage Strategy Pattern . . . 7
3.3.2 gRPC Server Implementation . . . 8
3.4 AIService: On-Premise LLM Anomaly Scoring . . . 8
3.5 Sentinel Gateway UI: Real-Time Security Operations . . . 8

4 DevOps, Cloud Infrastructure & CI/CD **9**
4.1 Kubernetes Production Helm Chart (`k8s/`) . . . 9
4.2 Terraform Infrastructure as Code (`terraform/`) . . . 9
4.3 Cross-Platform Automation Scripts (`scripts/`) . . . 9
4.4 GitHub Actions Quality Gates (`.github/workflows/`) . . . 10

5 Strategic Roadmap & Future Evolution **11**
5.1 Phase 1 — Kernel-Level eBPF / XDP Enforcement . . . 11
5.2 Phase 2 — RAG-Powered Threat Memory . . . 11
5.3 Phase 3 — Dynamic Honeytoken Trapping . . . 11
5.4 Phase 4 — Service Mesh & Envoy WebAssembly Filter . . . 11

§1 Executive Summary & System Scenarios

1.1 The Architectural Vision

Modern cloud-native microservice fabrics face a critical architectural challenge: traditional Web Application Firewalls (WAFs) operate on static regular expressions that fail against dynamic zero-day attacks, distributed enumeration, and API logic abuse. Conversely, cloud-hosted AI security solutions introduce hundreds of milliseconds of network latency directly into the critical HTTP request loop and expose private corporate telemetry to external third parties.

SentientGate eliminates this compromise. It is an autonomous runtime security fabric designed to be deployed directly in front of microservice topologies. By decoupling immediate traffic routing from heavy threat evaluation via an **Out-of-Band (OOB) event-driven architecture**, SentientGate achieves:

- **Sub-Millisecond Edge Enforcement (<1ms):** Clean requests pass through the Spring Cloud Gateway without perceptible latency, while Redis in-memory lookups immediately drop blacklisted actors with `403 Forbidden`.
- **Asynchronous Kafka Telemetry:** Complete audit logging and security anomaly alerts are published to Apache Kafka (`USER_LOGS` and `SECURITY_EVENTS`) out-of-band.
- **Cryptographic Visitor Fingerprinting:** Tamper-proof HMAC-SHA256 signed HTTP-only cookies prevent malicious clients from resetting their tracking identities.
- **Isolated Concurrency Pipelines:** Synchronous heuristic strategies run on a high-throughput thread pool (`analysisExecutor`), completely separated from slow AI inference calls (`aiExecutor`).
- **Local, Zero-Egress LLM Inference:** Integrates with on-premise `Ollama` running open-source models (`gemma3.2`, `llama3`, `qwen2.5`), guaranteeing that zero customer data leaves private infrastructure.
- **Dynamic TTL Blacklisting:** Mitigates false-positive impacts by applying dynamic, severity-calibrated Time-To-Live expiration keys in Redis (5m, 15m, 60m).
- **Turnkey Cloud-Native Orchestration:** Fully packaged with Kubernetes Helm charts (`k8s/`), Terraform AWS EKS Infrastructure as Code (`terraform/`), and automated multi-architecture Docker CI/CD pipelines.

1.2 Production Service Inventory

Service	Core Role & Architecture	Port
ApiGateway	Reactive reverse proxy (WebFlux), HMAC visitor signing, Redis rate-limiting and blacklist filtering	8080 / 8079
MCPService	Malicious Client Protection engine; multi-strategy evaluation, gRPC client, isolated thread pools	9991
AIService	Non-blocking WebFlux service querying local Ollama LLMs for behavioral anomaly scoring	8082
LoggingService	Dual-tier log consumer (PostgreSQL + Redis cache), gRPC Protobuf history server, metrics REST API	8010 / 9090
EurekaServer	Netflix Eureka registry for dynamic discovery and client-side load balancing	8761
DummyService	Protected downstream service demonstrating traffic routing and security enforcement	8090
sentinel-ui	Real-time React 18 / Vite / Tailwind monitoring dashboard with live pipeline visualization	5173

§2 End-to-End Architectural Lifecycle

2.1 Request & Threat Detection Lifecycle

1. **Ingress & Identity Resolution:** The client request hits ApiGateway. VisitorIdFilter inspects the SENTINEL_VISITOR_ID cookie. If absent or invalid, it cryptographically signs a new UUID with HMAC-SHA256 and sets a secure HTTP-only cookie.

2. **In-Memory Blacklist Interception:** BlacklistFilter performs a non-blocking reactive check against Redis: blacklist:{uuid}. If present, the gateway immediately returns 403 Forbidden.

3. **Rate Limiting & Upstream Forwarding:** The gateway evaluates Redis Token Bucket rate limits (RequestRateLimiter) and routes clean traffic to the target microservice (e.g., DummyService).

4. **Out-of-Band Audit Telemetry:** SentientGateFilter captures request latency and status code, publishing a LogEvent to Kafka topic USER_LOGS.

5. **Security Event Triggering:** If the response status is an error ($4xx$ or $5xx$), a SecurityAlertEvent is simultaneously dispatched to Kafka topic SECURITY_EVENTS.

6. **Dual Ingestion in LoggingService:** Two independent Kafka consumers process USER_LOGS:
   - RedisLogConsumer: Caches the log into Redis list log:recent:{uuid} in ~1ms.
   - DatabaseLogConsumer: Flushes records in batches into PostgreSQL for permanent audit trails.

7. **Batch Consumption & Deduplication in MCPService:** MCPService consumes SECURITY_EVENTS in batches, groups alerts by client UUID, and filters out duplicates via RedisGuardService using atomic Redis SETNX keys.

8. **Fast Binary gRPC History Retrieval:** MCPService calls LoggingService over a high-speed gRPC Protobuf stream (user_log_event.proto), pulling the client's past 10 minutes of activity.

9. **Layered Synchronous Strategy Execution:** Offloaded to the analysisExecutor pool, heuristics run in strict priority order (Pattern Match, Sensitive Path, Rate-Limit Cooldown, High Error Rate, Burst Traffic).

10. **Asynchronous LLM Anomaly Scoring:** If heuristics do not trigger a block, the event history is dispatched to the aiExecutor pool, triggering AIService which prompts local Ollama LLMs.

11. **Dynamic Enforcement Execution:** If malicious intent is established, MCPService writes a JSON block record to Redis with an automated TTL, instantly blocking the client from the entire microservice ecosystem.

2.2 Concurrency & Thread Isolation Model

To guarantee resilience under heavy DDoS attack scenarios, SentientGate combines reactive event loops with strictly partitioned worker pools:

[*] ARCHITECTURAL CAPABILITY

1. Gateway & AI Layer (Netty Event Loops): Both ApiGateway and AIService are built on Spring WebFlux. They operate on non-blocking Netty event loops without allocating a thread per connection. I/O operations (Redis checks, Kafka producer dispatches, and HTTP calls) never block thread execution.
2. Security Engine Isolation (MCPService): Defined in AsyncConfig.java, two dedicated thread pools isolate workloads:

- `analysisExecutor` (Core: 8, Max: 16, Queue: 256): Executes fast heuristic rules. Equipped with `CallerRunsPolicy` to apply natural backpressure to Kafka consumers if saturated.
- `aiExecutor` (Core: 4, Max: 8, Queue: 128): Dedicated strictly to asynchronous Ollama LLM requests. Ensures slow or stuttering AI models never starve rule analysis.

§3 Microservice Component Specifications

3.1 ApiGateway: Non-Blocking Edge Mesh

ApiGateway is the single entry point for all external traffic. It provides intelligent routing, authentication verification, cryptographic tracking, and edge blocking.

3.1.1 Filter Chain Execution Order

Order	Filter Component	Operational Function
1	VisitorIdFilter	Generates / validates HMAC-SHA256 signed cookie `SENTINEL_VISITOR_ID`; mutates exchange with validated UUID header
2	BlacklistFilter	Checks Redis key `blacklist:{uuid}`; returns `403` if present
3	JwtExtractionFilter	Validates Bearer JWTs, checks revoked token IDs (`jti`) against Redis, and forwards claims downstream
4	Upstream Routing	Evaluates Spring Cloud Gateway routes with Redis token-bucket rate limiting (`RequestRateLimiter`)
5	SentientGateFilter	Global post-filter: measures execution latency, publishes to Kafka `USER_LOGS`, and dispatches $4xx/5xx$ alerts to `SECURITY_EVENTS`

3.1.2 Management & Metrics Controllers

The gateway exposes management APIs for security operators:

- `GET /api/mgmt/blacklist`: Lists active blacklist keys, block reasons, and remaining TTLs.
- `POST /api/mgmt/blacklist`: Manually blocks a malicious UUID with custom TTL and reason.
- `DELETE /api/mgmt/blacklist/{uuid}`: Immediately unblocks a client.
- `GET /api/dashboard/metrics`: Exposes real-time throughput, active blocks, and service health metrics.

3.2 MCPService: Malicious Client Protection Engine

MCPService functions as the analytical security brain. It consumes security alerts, fetches behavioral histories over gRPC, executes heuristic strategies, and interfaces with the AI service.

3.2.1 Layered Heuristic Strategies

Threat Strategy	Inspection Logic	Enforcement Action
BurstTrafficStrategy	Evaluates total requests in a sliding window; detects request spikes exceeding threshold	**TTL: 15 min**
PatternMatchStrategy	Normalizes paths (URL decode, Unicode NFKC, SQL comment strip) and scans for SQLi, XSS, and Traversal	**TTL: 60 min**
SensitivePathStrategy	Scans for unauthorized probes targeting `.env`, `/actuator`, `/admin`, `/wp-login`	**TTL: 60 min**
RateLimitCoolDownStrategy	Detects clients repeatedly generating `429 Too Many Requests` violations	**TTL: 15 min**
HighErrorRateStrategy	Calculates client failure ratio; triggers if $> 50\%$ of recent requests result in errors	**TTL: 15 min**
AiAnomalyStrategy	Contextual multi-factor LLM assessment of route hops, IP switches, and timing anomalies	**TTL: 5–60 min**

3.2.2 Resilience & Fault Tolerance

External calls to the AI Service are wrapped using Resilience4j:

- Sliding window of 10 calls with a 50% failure threshold.
- 10-second wait duration in Open state with automatic half-open transitions.
- 2-second time limiter and 3 retry attempts for network disconnects.

3.3 LoggingService: Persistence & gRPC History Provider

LoggingService consumes the high-throughput `USER_LOGS` Kafka topic and exposes historical logs to MCPService.

3.3.1 Storage Strategy Pattern

To resolve the trade-off between database write overhead and sub-millisecond retrieval, LoggingService implements a LogFetchStrategyResolver:

- RedisFetchStrategy: First checks Redis list `log:recent:{uuid}`, returning logs in sub-milliseconds.
- DatabaseFetchStrategy: Falls back to PostgreSQL table `gateway_logs` on cache miss or for multi-hour audits.

3.3.2 gRPC Server Implementation

Implemented in UserLogEventGrpcService.java, the gRPC service serves protobuf-serialized request histories (`user_log_event.proto`) on port `9090`, slashing internal network serialization overhead by $> 60\%$ compared to standard REST JSON.

3.4 AIService: On-Premise LLM Anomaly Scoring

AIService is a WebFlux microservice connecting directly to a local Ollama instance running on the host or in a cluster pod:

- **Dynamic Feature Engineering:** Calculates failure rates, request velocity per minute, route entropy, and IP hop counts from the incoming history list.
- **Defensive Prompt Formatting:** Injects strictly validated numerical and structural features into an engineered system prompt, forcing the LLM to output pure JSON containing threat scores (0.0 to 1.0) and forensic justifications.
- **Safe Fallback Parsing:** Enforces strict 5-second timeouts and numeric fail-safe parsing. If the LLM stutters or times out, it safely falls back to a score of `0.0`, preventing false positives.

3.5 Sentinel Gateway UI: Real-Time Security Operations

The dashboard (`UI/sentinel-gateway-ui`) is built with **React 18, TypeScript, Vite, and Tailwind CSS**:

- **Live Metric Gauges:** Displays requests per minute, blocked threat counts, P99 gateway latency, and microservice instance status.
- **Architectural Pipeline Flow:** Interactive visual diagram representing live traffic streaming between Gateway, Kafka, MCP, and AI services.
- **Threat Intelligence Ledger:** Live table of blocked UUIDs showing blocking reasons, severity levels, active TTL countdowns, and manual unblock actions.
- **User Investigation Drawer:** Allows operators to click any UUID to inspect their historical request timeline, status distribution, and behavioral attributes.

§4 DevOps, Cloud Infrastructure & CI/CD

4.1 Kubernetes Production Helm Chart (k8s/)

SentientGate packages its entire cloud deployment into a production-ready Helm chart:

- **Microservice Deployments:** Declarative deployments, Services, and Ingresses for api-gateway, mcp-server, ai-service, logging-service, eureka-server, and sentinel-ui.
- **Stateful Infrastructure:** Pre-configured manifests for Apache Kafka (in KRaft mode), Redis, PostgreSQL, and Kafka-UI.
- **Ingress Controllers:** NGINX Ingress configurations exposing public endpoints while shielding internal gRPC and MCP ports from external access.

4.2 Terraform Infrastructure as Code (terraform/)

The terraform/ module provisions an enterprise-grade AWS environment:

- **AWS VPC Module:** Provisions a dedicated VPC with public and private subnets across multiple Availability Zones, equipped with NAT Gateways for secure egress.
- **Amazon EKS Cluster:** Deploys a managed Kubernetes cluster (v1.31) with autoscaling node groups and IAM OIDC integrations configured for GitHub Actions CI/CD deployment roles.

4.3 Cross-Platform Automation Scripts (scripts/)

Automated operations are maintained across operating systems in scripts/linux/, scripts/macos/, and scripts/windows/:

- run_local.*: Spins up Docker infrastructure (Postgres, Redis, Kafka), validates health checks, boots Eureka, and launches all microservices.
- stop_local.*: Gracefully terminates local service processes and Docker containers.
- test_local.*: Sequentially executes integration and unit test suites across all Maven modules and frontend packages.
- build_and_push_images.*: Compiles native Docker containers and publishes images to Docker Hub.
- generate_protos.*: Compiles Protobuf schemas into generated Java gRPC classes.

4.4 GitHub Actions Quality Gates (`.github/workflows/`)

- **PR Validation Pipeline (`pr-validation.yml`):** Automatically triggers on pull requests targeting `main`. Boots containerized PostgreSQL, Redis, and Kafka services to run complete automated test suites across all services. Outdated PR runs are automatically canceled.
- **Main CI/CD Pipeline (`ci-cd.yml`):** Triggers on direct pushes or merged PRs. Runs the full test suite first, and only on success builds and publishes multi-architecture Docker images to Docker Hub.
- **Cloud Release Deploy (`release-deploy.yml`):** Deploys updated Helm charts directly to Amazon EKS clusters via temporary, non-committed secrets.

§5 Strategic Roadmap & Future Evolution

5.1 Phase 1 — Kernel-Level eBPF / XDP Enforcement

Push Redis blacklist evaluations directly into the Linux kernel using eBPF and XDP (eXpress Data Path). Blacklisted packets will be dropped at the network interface card (NIC) level at wire speed, neutralizing volumetric DDoS before user-space context switches occur.

5.2 Phase 2 — RAG-Powered Threat Memory

Equip AIService with vector database capabilities using PostgreSQL's pgvector. Historical attack vectors and multi-day distributed reconnaissance traces will be converted into embeddings, enabling the LLM to identify distributed, low-and-slow campaigns spanning weeks.

5.3 Phase 3 — Dynamic Honeytoken Trapping

Introduce automated honeytokens that dynamically inject decoy routes (e.g., hidden '/api/v2/internal-keys') and false credentials into responses. Automated scrapers and fuzzers interacting with these honeytokens are immediately blacklisted with maximum severity.

5.4 Phase 4 — Service Mesh & Envoy WebAssembly Filter

Package SentientGate's edge filter as an Envoy WebAssembly (Wasm) plugin. This will allow the security fabric to deploy natively into Istio and Linkerd service meshes across multi-cloud enterprise topologies.